



UNIVERSIDAD INTERNACIONAL DEL ECUADOR — UIDE

Facultad de Ingeniería · Carrera: Ingeniería en Inteligencia Artificial

Gestor de Finanzas Personales

*Un programa de consola en Python aplicando estructuras
de control*

Asignatura	Lógica de Programación 1
Actividad	Aprendizaje Autónomo 2
Estudiante	Sebastian Cumba

1. Introducción

La mayoría de las personas administra su dinero "de memoria", sin un registro ordenado de cuánto entra y cuánto sale. Este vacío no es menor: la **alfabetización financiera** —la capacidad de entender y usar información económica para tomar decisiones— se asocia con mejor ahorro, menor endeudamiento y mayor bienestar (Lusardi & Mitchell, 2014). El problema, además, empieza temprano: en la evaluación internacional PISA 2022, en promedio el 18 % de los estudiantes de los países de la OCDE no alcanzó el nivel básico de competencia financiera (OECD, 2024). Existe, entonces, una necesidad real y medible a la que un programa sencillo puede responder.

Este trabajo presenta el **Gestor de Finanzas Personales**, un programa de consola escrito en **Python** que permite registrar ingresos y gastos, consultar un resumen y recibir alertas cuando el gasto de una categoría se acerca a su presupuesto. La **tesis** que sostiene el proyecto es que un problema cotidiano y socialmente relevante puede resolverse de forma completa empleando únicamente las **estructuras de control** estudiadas en el curso —secuencia, **condicionales** y **bucles**—, y que esa restricción, lejos de limitar, obliga a pensar la lógica con claridad. La conexión con la **ingeniería en inteligencia artificial** es directa: todo sistema inteligente se apoya en esta misma base lógica, y un gestor de gastos es el primer peldaño hacia un asistente que algún día podría *recomendar* y no solo *registrar*.

2. Fundamentación: por qué este problema y por qué así

Evidencia del problema. Lusardi y Mitchell (2014) tratan el conocimiento financiero como una forma de **inversión en capital humano**: quien lo posee toma mejores decisiones de ahorro e inversión. Es una postura sólida y respaldada por datos; sin embargo, conviene no tomarla como dogma. Su evidencia muestra *correlación*, y otros factores —ingreso, educación, contexto— influyen a la vez, por lo que una herramienta de registro ayuda, pero no sustituye a las condiciones materiales de cada persona. El informe de la OECD (2024) aporta el dato que motiva el enfoque juvenil del proyecto y, de paso, una pista de diseño: el desempeño en lectura y matemáticas se correlaciona fuertemente con la competencia financiera, lo que sugiere que una herramienta debe ser **simple y legible** para no excluir a quien más la necesita.

Evidencia del método. Que baste con tres estructuras no es una opinión, es un teorema. Böhm y Jacopini (1966) demostraron que cualquier programa puede escribirse combinando solo **secuencia, selección e iteración**, sin saltos arbitrarios (*goto*). Años después, Dijkstra (1968) llevó ese resultado a la práctica con su célebre crítica al *goto*, argumentando que el código estructurado es más fácil de leer y razonar. Estos dos trabajos justifican teóricamente la consigna del curso: organizar el programa con condicionales y bucles bien anidados no es un capricho académico, sino la forma demostradamente suficiente —y más mantenible— de

construir software. Para la implementación se siguió el enfoque práctico y por proyectos de Matthes (2023).

3. Arquitectura y funcionalidades

El programa separa la **lógica pura** (cálculos, sin teclado ni pantalla) de la **interacción** (menús y validación). Esta decisión permite probar la lógica de forma automática y mantener cada parte enfocada. Los módulos son: *finanzas.py* (registro y totales), *presupuesto.py* (reglas de alerta), *almacenamiento.py* (guardar/cargar en JSON), *util.py* (validación de entradas) y *main.py* (el bucle del menú). Las cuatro funcionalidades principales se diseñaron primero como **diagramas de flujo** y luego se tradujeron a código.

El corazón del programa es un **bucle while** que mantiene vivo el menú hasta que el usuario decide salir, y un **condicional** encadenado que despacha cada opción.

1. Bucle del menú principal

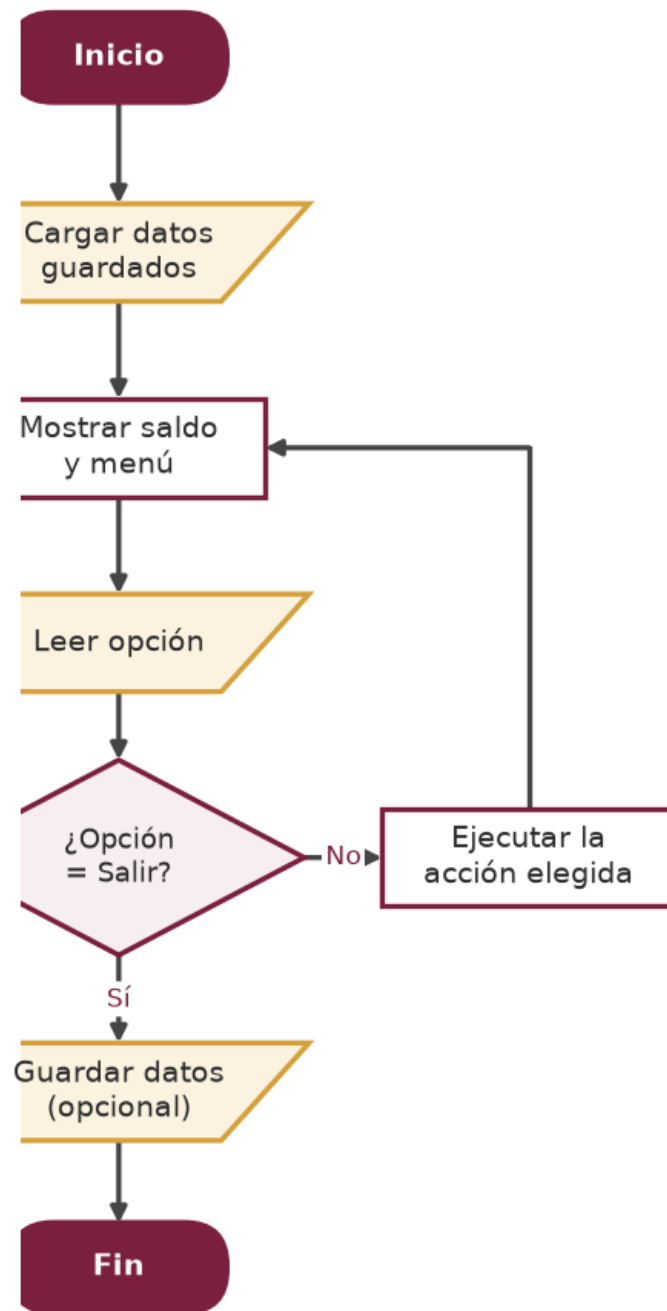


Figura 1. Diagrama de flujo del bucle del menú principal.

El registro de un movimiento muestra cómo la **validación de entradas** se apoya en un bucle de reintento: si el monto no es un número positivo, el programa no se cae, vuelve a preguntar.

2. Registrar ingreso o gasto

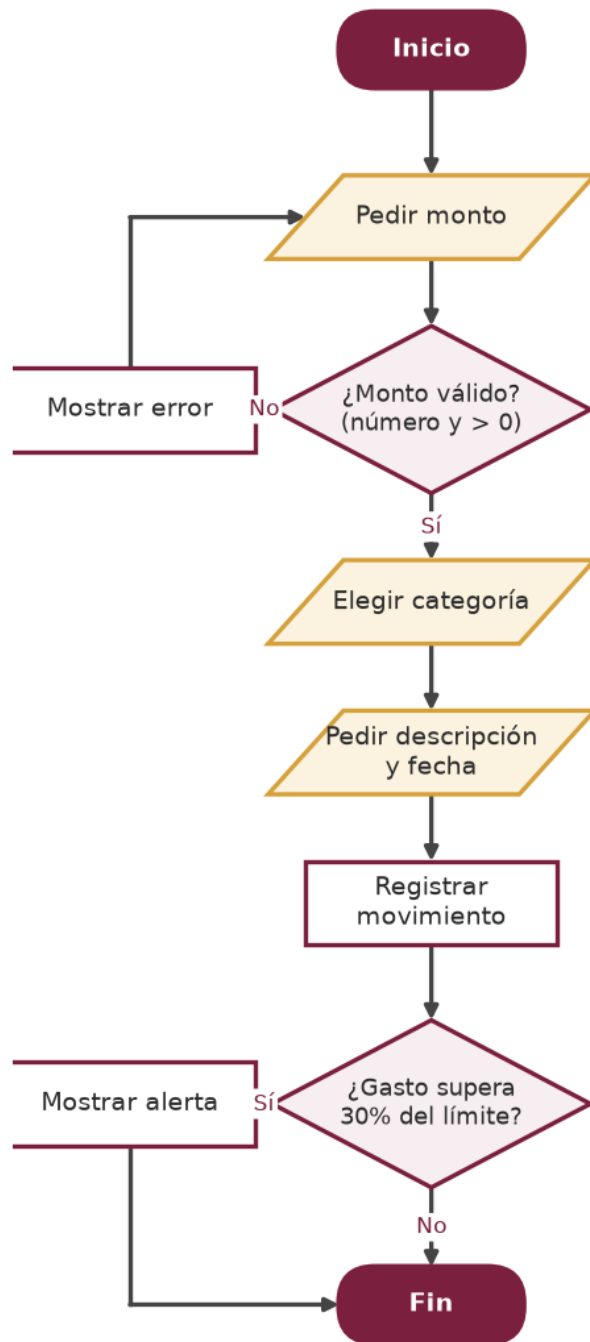


Figura 2. Diagrama de flujo del registro de un ingreso o gasto, con validación.

El resumen recorre todos los movimientos con un **bucle for** y va **acumulando** el gasto en cada categoría, decidiendo con condicionales a qué grupo pertenece cada uno.

3. Ver resumen (bucle por categoría)

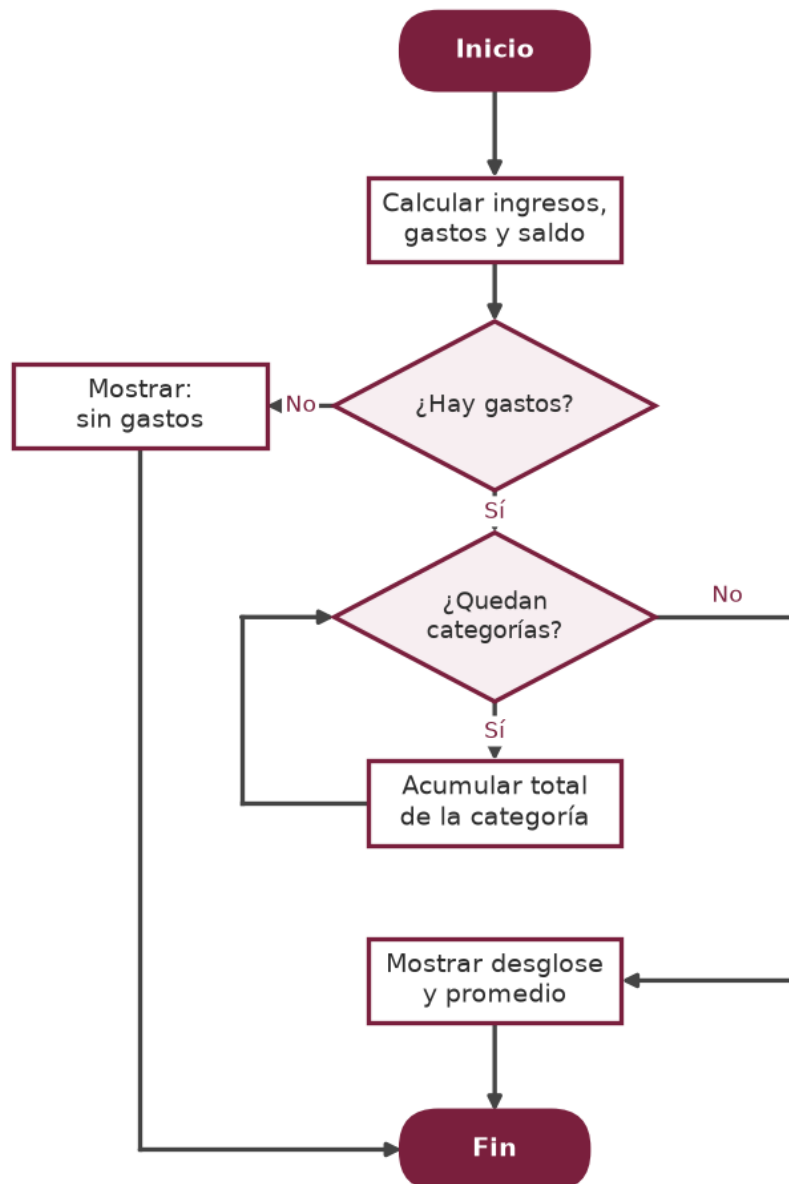


Figura 3. Diagrama de flujo del resumen por categoría.

Las alertas son el mejor ejemplo del **condicional escalonado** (*if / elif / else*): el porcentaje gastado se compara, de lo más grave a lo más leve, contra umbrales del 100 %, 80 % y 50 %, produciendo un semáforo de riesgo.

4. Semáforo de alertas (if / elif / else)

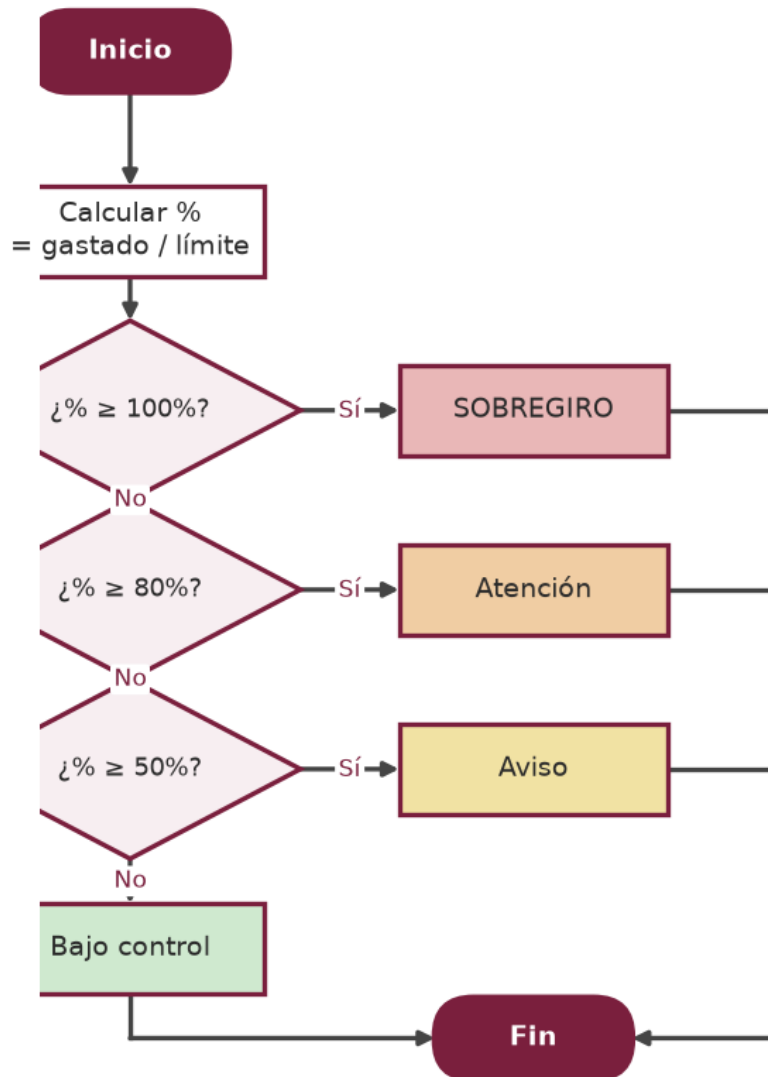


Figura 4. Diagrama de flujo del semáforo de alertas de presupuesto.

Ejemplo concreto y medible. Con los datos de prueba incluidos, el programa calcula un saldo de \$440.00 (ingresos de \$1 000.00 menos gastos de \$560.00). Al registrar un gasto adicional de \$60.00 en *Ocio*, el sistema dispara dos respuestas verificables: una alerta inmediata —ese gasto equivale al 60 % del presupuesto de la categoría— y, en el panel, la categoría *Ocio* pasa a estado "Atención" al alcanzar el 90 %, mientras *Vivienda* aparece en "Sobregiro" con el 100 %. La lógica, además, está cubierta por nueve **pruebas automáticas** que pasan satisfactoriamente.

4. Perspectivas alternativas y contradicciones

Ninguna decisión de diseño es la única correcta, y reconocerlo fortalece el trabajo. Una **aplicación web o móvil** sería más atractiva y portable que una de consola; se eligió la consola porque hace visible la lógica —el objetivo del curso— sin el "ruido" de interfaces gráficas. Del mismo modo, una **hoja de cálculo** resuelve buena parte del problema sin programar; la diferencia es que un programa permite **reglas activas** (alertas) y abre la puerta a la automatización, algo que una hoja estática no ofrece con la misma naturalidad.

Hay una contradicción más profunda, propia de la disciplina. El programa evita por completo el *goto* y se apoya en estructuras anidadas, siguiendo a Dijkstra (1968); pero esa misma postura fue —y en parte sigue siendo— discutida: estructuras como *break* o *continue*, que el proyecto usa en la validación, son "saltos controlados" que algunos puristas evitarían. El proyecto toma partido por la **legibilidad práctica** sobre la pureza teórica, asumiendo el costo de esa elección. Finalmente, el modelo de **presupuesto fijo** por categoría es una simplificación: los gastos reales fluctúan entre meses, y un umbral rígido puede generar falsas alarmas. Se acepta esta limitación a cambio de transparencia, y se señala como el punto natural de mejora.

5. Pensamiento innovador

Sobre la base de un registro de gastos —idea común— el proyecto agrega elementos propios. El **semáforo de alertas** convierte números fríos en una señal de comportamiento (verde, amarillo, naranja, rojo), y la **alerta por gasto puntual** advierte en el momento mismo de gastar, no al final del mes. La separación entre lógica pura e interacción habilitó un **modo demostración** (--demo) que ejecuta el programa sin intervención, útil tanto para las pruebas como para el video. Mirando hacia adelante, esa misma separación es lo que permitiría, sin reescribir el núcleo, incorporar una capa de **inteligencia artificial**: un modelo que aprenda los patrones de gasto del usuario y ajuste los umbrales automáticamente, transformando un gestor que *reacciona* en uno que *anticipa*.

6. Conclusiones

El proyecto confirma su tesis: un problema real y socialmente valioso se resolvió por completo con secuencia, condicionales y bucles, tal como anticipa el teorema de la programación estructurada (Böhm & Jacopini, 1966). La principal **implicación** es que el dominio de pocas estructuras bien combinadas habilita soluciones útiles; la principal **consecuencia práctica** es una herramienta que no solo informa, sino que advierte. El trabajo también deja una **síntesis**: unió un problema socioeconómico (Lusardi & Mitchell, 2014; OECD, 2024) con un fundamento de ciencias de la computación (Böhm & Jacopini, 1966; Dijkstra, 1968) en un único artefacto funcional, y de esa unión surgió una dirección nueva —

el salto del registro a la recomendación— que conecta el ejercicio con mi formación en inteligencia artificial.

Al terminar comprendí que la parte difícil no fue escribir el código, sino *pensar la lógica* antes de escribirlo: cada diagrama de flujo me obligó a decidir con antelación qué pasaría en cada caso. Esa disciplina de anticipar las decisiones es, creo, la misma que necesitaré para diseñar sistemas inteligentes, donde un descuido en la lógica se multiplica. Me llevo la idea de que la simplicidad bien pensada es una forma de potencia, no una limitación.

7. Bibliografía

- Böhm, C., & Jacopini, G. (1966). Flow diagrams, Turing machines and languages with only two formation rules. *Communications of the ACM*, 9(5), 366–371.
- Dijkstra, E. W. (1968). Go to statement considered harmful. *Communications of the ACM*, 11(3), 147–148.
- Lusardi, A., & Mitchell, O. S. (2014). The economic importance of financial literacy: Theory and evidence. *Journal of Economic Literature*, 52(1), 5–44.
- Matthes, E. (2023). *Python crash course: A hands-on, project-based introduction to programming* (3.^a ed.). No Starch Press.
- OECD. (2024). *PISA 2022 results (Volume IV): How financially smart are students?* OECD Publishing.